

TREE TRAINING: ACCELERATING AGENTIC LLMs TRAINING VIA SHARED PREFIX REUSE

Jinghui Wang^{*†1} Shaojie Wang^{*1} Yinghan Cui^{*1} Xuxing Chen^{*1} Chao Wang^{*1} Liang Huang¹
Xiaojiang Zhang¹ Junyi Peng¹ Li Wan¹ Haotian Zhang¹ Bin Chen¹

ABSTRACT

Agentic large language model (LLM) training often involves multi-turn interaction trajectories that branch into multiple execution paths due to concurrent tool use, think-mode, sub-agent, context management and other runtime designs. As a result, the tokens produced by a single task naturally form a tree-structured token trajectory with shared prefixes, rather than a linear sequence. Existing training pipelines linearize such trajectories and treat each branch independently, leading to substantial redundant computation in both forward and backward passes. We derive that averaging the loss over all branches independently is algebraically identical to a per-token weighted loss, where each token’s weight equals the fraction of branches passing through it. The problem therefore reduces to computing the log-probability of every token in the prefix tree exactly once, with no repeated computation across shared prefixes: we propose DFS serialization of the tree, which visits every token exactly once, and adapt full-attention and SSM layers to ensure the resulting log-probabilities match independent per-branch calculation exactly. In practice, a single trajectory tree can be too large to fit in GPU memory; we therefore propose **Redundancy-Free Tree Partitioning**, which handles memory-constrained settings with zero redundant computation and peak memory bounded by a single root-to-leaf path. Together, these contributions form **Tree Training**, an efficient framework for training LLMs on tree-structured trajectories, achieving up to **6.2x** end-to-end training speedup on dense and MoE models for both supervised fine-tuning and reinforcement learning.

1 INTRODUCTION

Training Large Language Models (LLMs) in agentic scenarios involves multi-turn interactions with environments, where a single task frequently branches into multiple parallel paths. This branching is driven by tree-structured workflow designs (e.g., tree-structured planning (Yao et al., 2023; Zhang et al., 2024) and sub-agents (Team et al., 2026)), concurrent tool invocations (Kim et al., 2024; Abdelaziz et al., 2024), context management (Xu et al., 2025; Packer et al., 2024; Zhong et al., 2023), as well as runtime behaviors such as think-mode (discarding/replacing intermediate reasoning tokens between turns, e.g. Figure 1), and retokenization drift (Luo et al., 2025). These mechanisms imply that the context fed into turn $t+1$ is often *not* exactly the concatenation of the tokens produced in previous turns (from turn 1 to turn t); therefore, after finishing a task, the collected tokens cannot always be organized as a single list/tensor, but instead form a tree-structured token trajectory with shared prefixes across branches. As a result, training must process multiple sequences with substantial shared prefixes, lead-

ing to significant redundant forward/backward computation. Prefix caching (Kwon et al., 2023; Pope et al., 2022) addresses this at inference time by reusing identical KV states for the forward pass, but does not extend to training, where the gradient of each prefix token depends on all branches passing through it.

To address the above challenges, we mathematically derive that the baseline loss—averaging over all branches independently—is algebraically identical to a per-token weighted loss where each token’s weight equals the fraction of branches passing through it. The problem therefore reduces to computing the log-probability of every token in the prefix tree exactly once, with no repeated computation across shared prefixes. We propose DFS serialization of the tree, which visits every token exactly once, and adapt full-attention and SSM layers to ensure the resulting log-probabilities match the baseline exactly. An alternative approach is to process the tree one node at a time with differentiable context passing between nodes—this also achieves zero redundant computation but suffers from poor GPU utilisation due to many small forward calls and high kernel-launch overhead (analyzed in Section 3.3).

In practice, a single trajectory tree is often too large to fit entirely within GPU memory. We therefore propose

^{*}Equal contribution ¹Kwai Inc. Correspondence to: Jinghui Wang <wangjinghui05@kuaishou.com>.

g_n : # paths through n

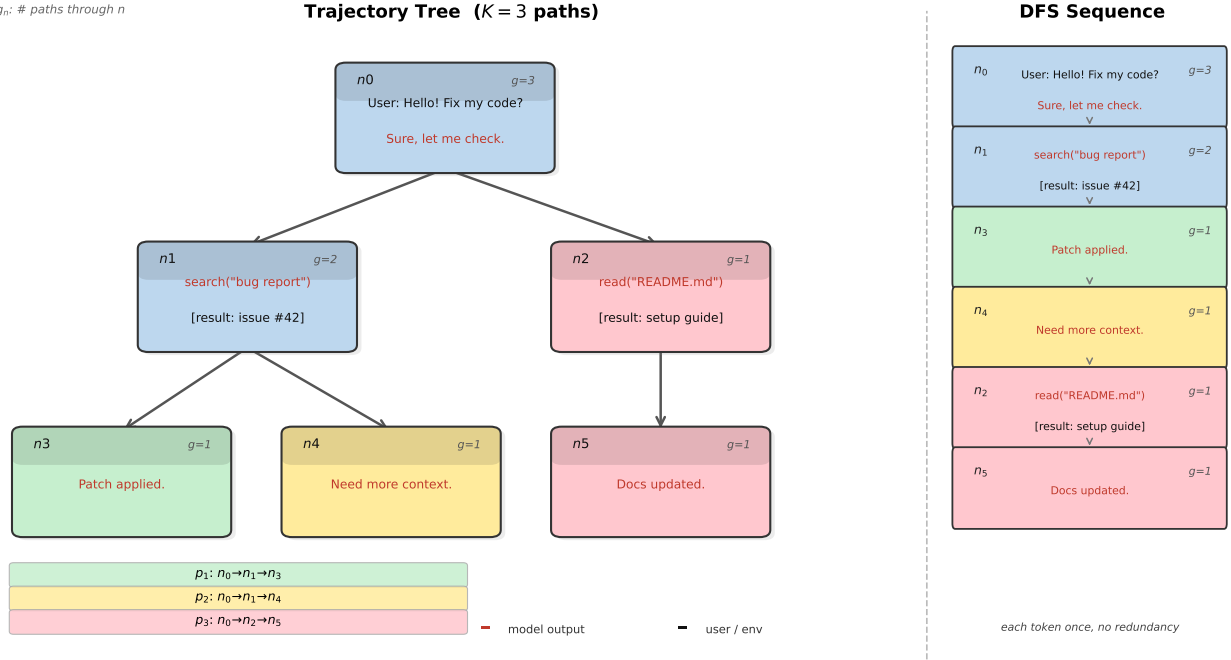


Figure 1. Trajectory tree and DFS sequence. Left: a trajectory tree with $K = 3$ root-to-leaf paths. Each node n holds a token segment T_n ; g_n is the number of paths through n (blue nodes are shared prefixes with $g_n > 1$). Node text in **red** is model output (trained); black is user/environment input. Right: the DFS sequence places all nodes in depth-first order so that every token is computed exactly once, with no redundancy.

Redundancy-Free Tree Partitioning: a hybrid that applies DFS serialization within memory-feasible subtrees for compute efficiency, while using differentiable partition boundaries to relay accumulated states (KV caches and, for SSM hybrid models, recurrent states) without ancestor re-computation, keeping peak memory bounded by a single root-to-leaf path.

Together, DFS serialization with model-layer adaptations and Redundancy-Free Tree Partitioning form **Tree Training**: an efficient framework for training LLMs on tree-structured trajectories for both SFT and RL, achieving up to **6.2×** end-to-end training speedup in our experiments.

In summary, our contributions are threefold:

- (1) We identify a pervasive yet overlooked property of agentic LLM training: trajectories naturally form overlapping tree-structured prefixes, creating substantial opportunities for computation reuse.
- (2) We introduce **Tree Training**, a paradigm that executes each shared prefix once while achieving mathematically equivalent training to the baseline, via DFS serialization, targeted model-layer adaptations, and—under memory constraints—**Redundancy-Free Tree Partitioning** with zero redundant computation and peak memory bounded by a single root-to-leaf path.

(3) We demonstrate consistent gains across dense and MoE (DeepSeek-AI et al., 2024) models on diverse datasets, achieving average 6.2× end-to-end speedup—and in agentic RL scenarios—without compromising training fidelity or final model quality.

2 RELATED WORK

Prefix caching is widely used to speed up autoregressive decoding by reusing identical prefixes in the forward pass (Kwon et al., 2023; Pope et al., 2022; Liu et al., 2024; Cheng et al., 2024; Yao et al., 2025; Cheng et al., 2025; Qin et al., 2025). For training objectives where the shared part is purely a prompt, batching strategies such as prefix grouping can also reuse the prompt prefix computation without affecting gradients (Liu et al., 2025; Wang & Hegde, 2024). In addition, Goru et al. (Goru et al., 2025) propose a custom-mask approach to deduplicate repeated reasoning tokens within a single linear conversation. However, it does not address prefix reuse across multiple trajectories, and its prefixes never aggregate loss from multiple branches. As a result, it is a special case of the agentic scenario where the backward pass remains standard.

Prior works have also recognized that agentic/RL generation often forms tree-structured rollouts, and exploit shared prefixes primarily to improve *rollout* efficiency, e.g., tree-search

style rollouts and prefix reuse (Hou et al., 2025; Li et al., 2025; Ji et al., 2025). However, the subsequent training stage is commonly executed on linearized samples: current training pipelines (Luo et al., 2025; Zhang et al., 2025; Zhou et al., 2025) typically decompose a trajectory tree into separate linear segments and treat each segment independently, causing shared prefixes to be repeatedly recomputed in both forward and backward passes.

Sequence packing (Krell et al., 2021) refers to concatenating multiple short sequences into a fixed-length training example to improve training efficiency. Our proposed Tree Partitioning method generalizes this idea: it not only concatenates sequences, but also introduces a strategy for efficiently segmenting overly long training data structured as a prefix tree (noting that a sequence is a special case of a prefix tree). Crucially, by combining Redundancy-Free Tree Partitioning, our method eliminates the boundary recomputation that would otherwise arise from re-including ancestor tokens in each child subtree, thereby ensuring each token is computed exactly once across the entire tree.

Recent hybrid LLM architectures interleave full-attention layers with State Space Model (SSM) layers. Gated Delta Net (GDN) (Yang et al., 2025c) is a recent SSM variant that improves over earlier SSM architectures such as Mamba (Gu & Dao, 2024) and Mamba2 (Dao & Gu, 2024) by incorporating the delta rule, achieving better associative recall and in-context learning. GDN is adopted as the SSM component in hybrid models such as Qwen3.5 (Team, 2026). Unlike full-attention layers, SSM layers maintain a recurrent hidden state that is passed sequentially between tokens, which requires dedicated handling when processing tree-structured data.

Overall, unlike our work, these methods primarily target inference-time caching, prompt-only reuse, or single-conversation masking, and do not address the general case where a shared prefix must aggregate gradient contributions from multiple continuations.

3 TREE TRAINING

3.1 Derivation

A **trajectory tree** is a rooted tree in which each node n holds a token segment T_n . Each root-to-leaf path spells out a complete trajectory by concatenating the token segments of its nodes in order. Let $\mathcal{T}$ denote the set of all nodes and $\mathcal{P} = \{p_1, \dots, p_K\}$ the set of K root-to-leaf paths (one per leaf). An example with $K = 3$ is shown in Figure 1. **Goal.** Process all tokens in the tree with each token computed *exactly once* (no redundant computation across shared prefixes) in a single forward and backward pass, and produce gradients that are mathematically equivalent to the baseline of running all K paths independently and averaging their

losses.

The baseline sep-avg loss runs each of the K paths independently:

$$L_{\text{sep.avg}} = \frac{1}{K} \sum_{k=1}^K \sum_{t \in p_k} \ell_t(\theta) \quad (1)$$

where $\ell_t(\theta)$ is any per-token objective. For SFT, $\ell_t(\theta) = -\log p_\theta(y_t \mid x_{\leq t})$; for RL with policy gradient, $\ell_t(\theta) = -A_t \cdot \log p_\theta(y_t \mid x_{\leq t})$ where A_t is the per-token advantage.

Eq. (1) iterates over paths then over tokens in each path. We swap the order: iterate over every unique token t and count how many paths include it. Token t belongs to node n ; let $g_n = |\{p \in \mathcal{P} : n \in p\}|$ be the number of paths passing through n , and $g_t \triangleq g_n$ for $t \in T_n$. Token t is therefore counted g_n times in the double sum:

$$\sum_{k=1}^K \sum_{t \in p_k} \ell_t(\theta) = \sum_{n \in \mathcal{T}} \sum_{t \in T_n} g_n \cdot \ell_t(\theta) = \sum_t g_t \cdot \ell_t(\theta) \quad (2)$$

Substituting (2) into (1):

$$L_{\text{sep.avg}} = \sum_t \frac{g_t}{K} \cdot \ell_t(\theta) \quad (3)$$

We therefore define the tree loss:

$$L_{\text{tree}} = \sum_t \frac{g_t}{K} \cdot \ell_t(\theta) \quad (4)$$

By linearity of differentiation:

$$\frac{\partial L_{\text{tree}}}{\partial \theta} = \sum_t \frac{g_t}{K} \cdot \frac{\partial \ell_t}{\partial \theta} = \frac{\partial L_{\text{sep.avg}}}{\partial \theta} \quad (5)$$

This derivation is model-agnostic and objective-agnostic: since it relies only on the additive structure of ℓ_t across tokens and paths—not on its specific form—it applies to any per-token objective. The coefficient g_t/K is merely one special case arising from equal path weights ($w_k = 1/K$), chosen here to simplify the presentation; in general, any path weights $\{w_k\}$ yield a per-token weight $\lambda_t = \sum_{k: t \in p_k} w_k$, and the same conclusion holds: the loss reduces to a single weighted sum over unique tokens $\sum_t \lambda_t \cdot \ell_t(\theta)$, achievable in one forward pass by multiplying each token’s loss term by λ_t —without any change to how ℓ_t is computed. For example, setting $\lambda_t = 1$ for every unique token gives a valid training objective where each token contributes equally regardless of branch count; it is a distinct objective from sep-avg but equally straightforward to implement. Note that L_{tree} and $L_{\text{sep.avg}}$ are identical expressions—the substitution is purely algebraic; it does not yet impose any constraint on how $\ell_t(\theta)$ is computed in practice.

Problem reduction. The goal therefore reduces to: find a method that (1) computes each token *exactly once* with no

SSM State Routing

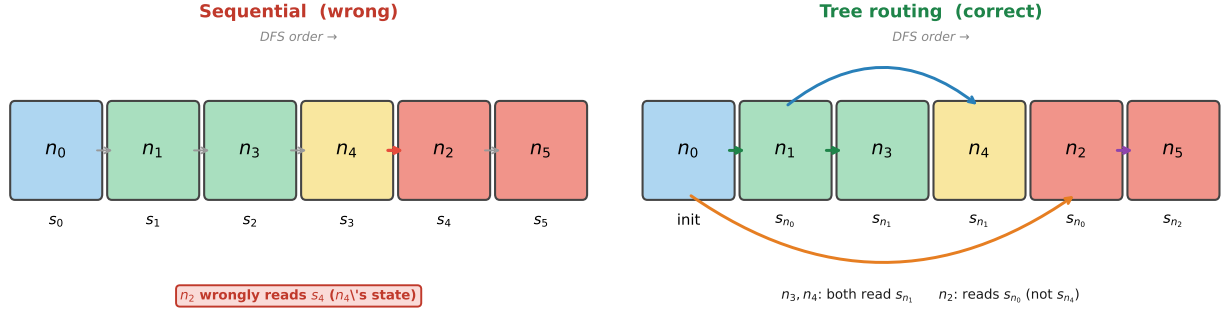


Figure 2. **SSM state routing.** Sequential flow (left) feeds n_4 's state into n_2 —wrong, since n_2 is a sibling of n_1 , not a descendant of n_4 . Tree routing (right) routes each chunk to its parent's state: both n_3 and n_4 read s_{n_1} , while n_2 correctly reads s_{n_0} .

redundant computation, and (2) ensures that $\ell_t(\theta)$ computed this way matches what it would be in a standalone per-branch forward pass—i.e., **forward equivalence**:

$$\ell_t^{\text{tree}}(\theta) = \ell_t^p(\theta) \quad \forall p \in \text{paths}(n_t) \quad (6)$$

We achieve this via DFS traversal of the token prefix tree, which serializes all nodes in depth-first order so each token appears exactly once, replacing the baseline serialization that repeats prefixes:

$$X_{\text{base}}(i) = \text{Concat}[\text{token}(i), X_{\text{base}}(\text{child}(0, i)), \text{token}(i), X_{\text{base}}(\text{child}(1, i)), \dots] \quad (7)$$

with:

$$X_{\text{DFS}}(i) = \text{Concat}[\text{token}(i), X_{\text{DFS}}(\text{child}(0, i)), X_{\text{DFS}}(\text{child}(1, i)), \dots] \quad (8)$$

How to enforce (6) on the **DFS-serialized** sequence—i.e., ensuring each token's log-probability matches its per-branch value despite the reordering introduced by DFS traversal—is described as follows.

3.2 Implementation

Attention Mask. We construct a tree attention mask such that token i attends to token j iff (1) $j \leq i$ and (2) the nodes of i and j lie on a common root-to-leaf path. This prevents cross-branch attention. We implement a GPU kernel based on Flash Attention V3 (Shah et al., 2024; Wang et al., 2025) with node-level shared-prefix masking. The attention pattern is illustrated in Figure 3.

Position Embedding. Each token must receive the same position ID as it would in its standalone per-path sequence.

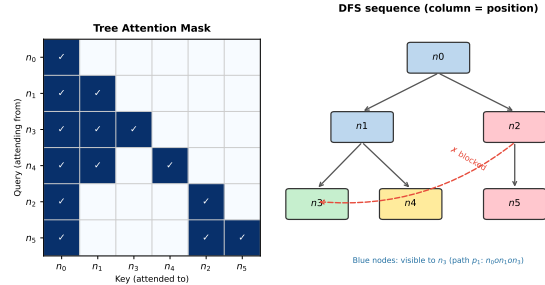


Figure 3. **Tree attention mask.** Left: the 6×6 attention matrix for the DFS sequence; a cell is filled iff the query token can attend to the key token. Tokens from different branches (e.g. n_3 and n_2) are mutually blocked. Right: the corresponding tree structure; the dashed red arrow shows a blocked cross-branch path.

A token at offset j within node n is assigned:

$$\text{pos}(t) = \sum_{n' \in \text{anc}(n) \setminus \{n\}} |T_{n'}| + j \quad (9)$$

where $\text{anc}(n)$ denotes the ancestors of n . Sibling nodes at the same depth share the same position range, matching the per-path baseline and ensuring RoPE (Su et al., 2023) produces identical results.

SSM Layers (GDN). For hybrid models that interleave full-attention with Gated Delta Net (GDN) (Yang et al., 2025c) SSM layers, the attention mask alone is insufficient: GDN layers maintain a recurrent hidden state passed sequentially between chunk boundaries, so a naïve DFS layout would carry the state from one branch into the next sibling branch. We apply two fixes. We treat each tree node's token segment as one *chunk*—the unit of SSM state transfer. (i) **State routing**: each chunk c reads its initial SSM state from its *parent* chunk $\pi(c)$ rather than the preceding chunk

in DFS order:

$$\mathbf{h}_c^{(0)} = \mathbf{h}_{\pi(c)}^{(\text{end})}, \quad \mathbf{h}_{-1}^{(\text{end})} \triangleq \mathbf{0} \quad (10)$$

DFS order guarantees $\pi(c) < c$, so parent states are always ready. Sibling chunks share the same parent state tensor, so their gradients accumulate there automatically during the backward pass. Figure 2 illustrates the difference between sequential and tree state routing.

(ii) **Causal convolution:** SSM layers include a causal conv1d with kernel size K_{conv} , which requires the preceding $K_{\text{conv}} - 1$ tokens as left context. In the DFS sequence, the tokens immediately before a child chunk belong to a sibling branch rather than the ancestor path, so using them as context is incorrect. Instead, each chunk c uses $\text{ctx}_{\pi(c)}$ —the last $K_{\text{conv}} - 1$ output tokens of its parent chunk $\pi(c)$, saved after processing the parent—as its left context:

$$\mathbf{y}_c = \text{Conv}([\text{ctx}_{\pi(c)}; \mathbf{x}_c])_{[K_{\text{conv}}-1:]} \quad (11)$$

This replicates the context each token would see in an independent per-branch forward pass. Figure 4 illustrates this for $K_{\text{conv}} = 3$.

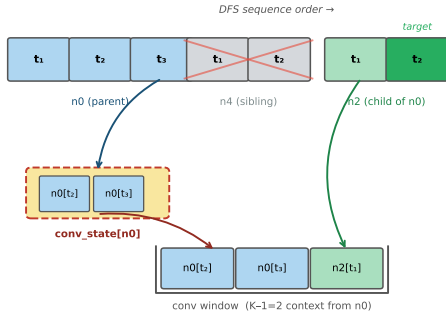


Figure 4. **SSM causal convolution** ($K_{\text{conv}} = 3$). When processing n_2 (child of n_0), the correct conv window is formed by prepending $\text{conv_state}[n_0]$ —the stored last $K_{\text{conv}} - 1$ effective tokens of the parent—as left context, bypassing the DFS-adjacent sibling tokens (n_4) that occupy the preceding positions.

Loss Weights. For each token t in the DFS-serialized sequence, its per-token loss $\ell_t(\theta)$ is multiplied by a weight λ_t before reduction. The specific value of λ_t depends on the training objective: for the sep-avg baseline with equal path weights this gives $\lambda_t = g_t/K$ as derived in Eq. (4); in general λ_t can be any value determined by the path weighting scheme (see Section 3.1). This is applied as a single element-wise multiplication on the per-token loss tensor—requiring no modification to the backward pass and negligible overhead. Full implementation details and float32 gradient verification appear in Appendix A.

3.3 Redundancy-Free Tree Partitioning

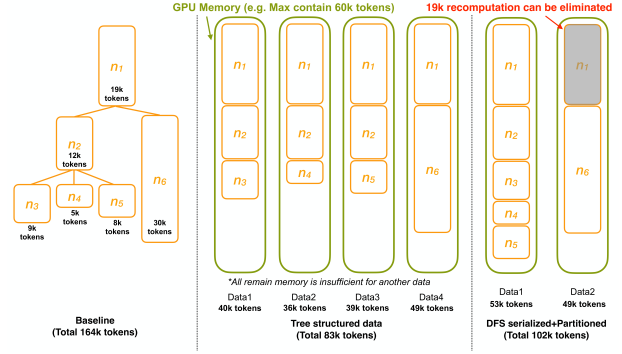


Figure 5. Memory-constrained training on a tree with 83k unique tokens (GPU limit $C = 60k$). Baseline flattening: 164k tokens. Standard Tree Partitioning (no differentiable boundaries): 102k tokens due to boundary recomputation. With differentiable partition boundaries, boundary recomputation is eliminated entirely: 83k tokens, equal to the tree’s unique token count.

When N_{tree} exceeds GPU memory capacity, the tree must be split into subtrees, each processed as a single DFS forward pass. Each subtree should be as large as GPU memory allows: larger subtrees pack more tokens into each forward call, improving compute density and reducing the number of model invocations (see the efficiency analysis in the last paragraph of this section).

Two design questions arise: (1) how to partition the tree optimally while keeping peak activation memory bounded; and (2) how to avoid redundant computation at partition boundaries—adjacent subtrees share a common prefix, and without a dedicated mechanism each child partition re-includes those shared ancestor tokens in its own DFS sequence, introducing redundant computation (e.g. Figure 5).

Partitioning. Partitions must follow tree node boundaries: each partition is a connected subtree, and every cut occurs at a node boundary. The reason is peak activation memory. The backward pass chains gradients bottom-up through the partition dependency graph; to bound peak memory at $\mathcal{O}(\text{max-path tokens})$, this graph must itself be a tree—meaning each partition has exactly one parent partition. This holds if and only if cuts are at node boundaries. Arbitrary cuts would place tokens from independent subtrees into the same partition; its backward must then simultaneously retain the computation graphs of *multiple* parent partitions (and transitively their ancestors), potentially keeping the graphs of all ancestor subtrees of all branches in memory at once.

The optimization goal is therefore to minimise $N_{\text{partitions}}$ —equivalently, maximise token utilisation of each partition—subject to each partition containing at most C tokens. This is a standard bin packing problem on tree subgraphs, solvable

with general-purpose combinatorial solvers such as OR-Tools (Perron & Furnon).

Differentiable partition boundaries. The second problem—avoiding redundant recomputation of shared prefixes across subtrees—is solved by making partition boundaries differentiable. At each cut node n_c , the parent partition’s accumulated KV tensors (and SSM states for hybrid models) are detached and re-exposed as differentiable leaf variables; the child partition attends to these cached ancestor key-value pairs without re-executing any ancestor token. After the child partition’s backward, gradients in the leaf tensors are passed back into the parent partition’s retained computation graph via an explicit backward pass through the parent—the same gradient-relay mechanism used in pipeline-parallel training—so each token is computed exactly once and total computation equals N_{tree} (Figure 5: 102k $\rightarrow$ 83k). For SSM hybrid models, the gateway additionally carries the recurrent hidden state and causal-conv context at the cut node, chained back identically. Details (ancestor-aware attention mask, depth-based position offset, float32 accumulation for multi-child branching, SSM state injection) are listed in Appendix B.

DFS packing versus per-node processing. The differentiable boundary mechanism above can be applied at *every* tree node boundary—not just partition boundaries—yielding *per-node processing*: the model is called N_{nodes} times, each processing one node’s own token segment, with the accumulated KV relayed from the parent node. Both approaches achieve zero redundant computation; DFS packing is preferable for two reasons. First, DFS processes all N_{tree} tokens in $\mathcal{O}(N_{\text{partitions}})$ calls instead of N_{nodes} , directly reducing CPU/kernel-launch overhead. Second, each DFS call covers far more tokens, making operations such as linear projections (GEMMs) significantly denser: in per-node processing, each call reloads the weight matrix with only the current node’s tokens to amortise it, while DFS amortises the same weight load over the entire partition. For attention, the same principle holds: in per-node processing, attention intensity is bounded by the node token count with no fixed cost to amortise, while DFS processes the full tree depth in one call. For SSM hybrid models, in per-node processing the recurrent state is written to and reloaded from HBM at every node boundary; DFS keeps it on-chip throughout the partition, avoiding the dominant memory bottleneck of SSM layers entirely (Gupta et al., 2026).

3.4 Does Tree Training introduce bias in the gradient updates?

Tree Training does not introduce bias during training. Each global batch is a self-contained tree generated from a single

rollout—the prefix-tree structure arises naturally within one task, so tree partitioning is organized and processed entirely within one gradient accumulation step, never mixing trajectories from different global batches. The data shuffle operation is applied only between complete tree samples and never disrupts the internal tree structure.

4 EXPERIMENTS

We evaluate Tree Training along three dimensions: gradient correctness, efficiency under realistic memory constraints, and end-to-end training performance. Across all settings, reusing shared prefixes from overlapping trajectories substantially reduces redundant computation while preserving equivalence to standard sequence-based baselines.

4.1 Metrics

To clearly separate dataset structure from runtime effects, we define the following metric.

Potential Overlap Ratio (POR) quantifies the theoretical upper bound of computation reuse by measuring the redundancy inherent in shared prefixes. We define POR as the ratio of overlapping tokens to the total token count of the flattened dataset:

$$\text{POR} = 1 - \frac{N_{\text{tree}}}{N_{X_{\text{base}}(\text{root})}} \quad (12)$$

Here, N_{tree} denotes the token count in the prefix tree, while $N_{X_{\text{base}}(\text{root})}$ represents the token counts of the serialized tree produced by the baseline method described in Equation (1). Using the example from Figure 5, the POR is calculated as $1 - \frac{83\text{k}}{164\text{k}} \approx 49.4\%$, indicating that nearly half of the baseline computation is redundant. Since Redundancy-Free Tree Partitioning computes each token exactly once regardless of memory constraints, the theoretical speedup upper bound is $1/(1 - \text{POR})$.

4.2 Evaluation Setups

We evaluate the end-to-end training efficiency by comparing the total time required to process equivalent data between Tree Training and the baseline described in Section 3. This baseline aligns with the current standard practice for training on branched trajectories, as most existing training frameworks only support taking batch of **list** (tensor) as training inputs.

All experiments were conducted on clusters of 64 NVIDIA Hopper GPUs using Megatron-Core (Shoeybi et al., 2020) for distributed training. Sequence Packing (Krell et al., 2021) is used in our baseline setting, representing the standard batching strategy used in large-scale LLM fine-tuning and RL training.

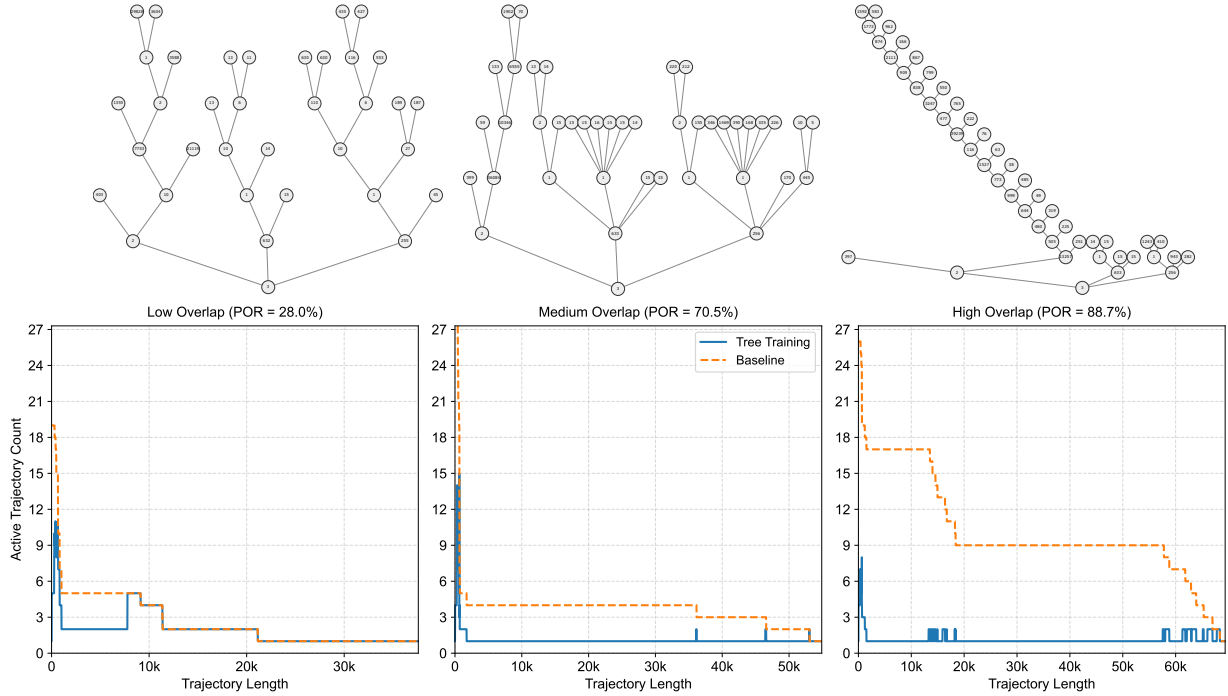


Figure 6. **Real agentic trajectory trees and their overlap characteristics.** The upper row shows representative trees from multi-turn agentic RL rollouts with **Low**, **Medium**, and **High Overlap**. The lower row plots *active trajectory counts* over length; the area ratio reflects the theoretical token reuse ratio.

4.3 Numerical Precision

While the derivation establishes exact mathematical equivalence, hardware floating-point arithmetic introduces unavoidable small numerical deviations in practice. The DFS-serialized forward pass accumulates intermediate results in a different order than independent per-branch passes, and cuBLAS may select different GEMM kernels for different tensor shapes, both causing non-associativity errors inherent to finite-precision floating-point arithmetic. These are not implementation bugs but fundamental hardware-level effects, and as shown in the next section, they do not affect training dynamics in practice.

4.4 Speedup and Correctness on real-world agentic trajectories

To assess real-world applicability, we collect agentic RL rollouts from multi-turn reinforcement learning tasks. Figure 6 presents representative realistic agentic trajectory trees (top) alongside their corresponding token-overlap distributions (bottom). These trajectories are collected from software engineering tasks (SWE-smith (Yang et al., 2025b)), executed via agentic scaffolds Claude Code (cc). Most branching behaviors of the left two trees are caused by concurrent tool execution and retokenization drift (Luo et al., 2025), while the right tree is caused by the think mode. These examples illustrate a broad spectrum of prefix sharing, ranging

from low to high POR (from 28.0% to 88.7%), and reveal that real agentic datasets tend to exhibit sparse, unbalanced trees with highly variable reuse across different depths and branching factors.

To validate the correctness and speedup of our method, we illustrate the loss comparison and step-time comparison on real-world agentic trajectories as shown in Figure 7:

(1) Speedup (Top Panel): The theoretical speedup upper bound $\frac{1}{(1-\text{POR})}$ for this dataset is **6.5x**. We achieved average realized speedups of **6.3x** (32B Dense) and **6.2x** (30B MoE). This demonstrates that our system incurs negligible overhead, capturing over **95%** of the theoretical potential. The step-wise fluctuation (2x–10x) faithfully reflects the varying intrinsic POR of dynamic tree samples.

(2) Correctness (Bottom Panel): Tree Training reproduces the baseline’s loss and gradient behaviors, with numerical deviations consistently maintained within acceptable precision tolerance. The training loss curves for Baseline (Yellow) and Ours (Green) overlap perfectly. The deviation (Red dashed) is negligible ($\leq 1\%$), confirming strict mathematical equivalence.

4.5 Speedup of datasets with varying POR

For controlled experiments, we construct synthetic datasets with POR values from 20% to 92%, while keeping the num-

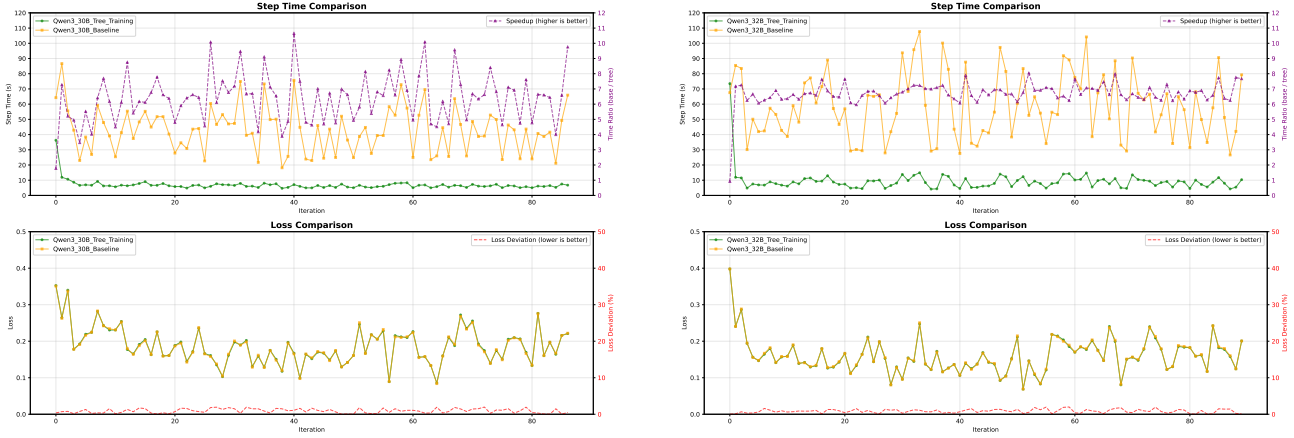


Figure 7. End-to-end training speedup and loss comparison using real world rollout data with think mode turned on. Results for the MoE model Qwen3-30B (Yang et al., 2025a) are shown on the left, and for the dense model Qwen3-32B on the right. For each model, the top plot illustrates the training speedup achieved by Tree Training, while the bottom plot displays the average relative error in loss.

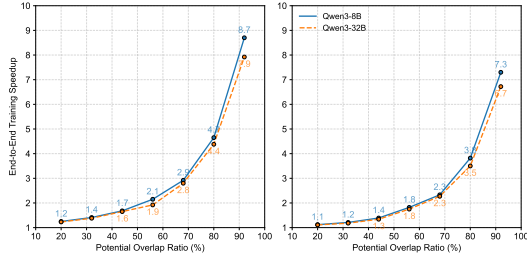


Figure 8. End-to-end training speedup of Tree Training across datasets with varying POR. Each subfigure reports the relative reduction in total training time of Tree Training compared to the baseline. (a) synthetic datasets where the full tree fits in GPU memory, (b) synthetic datasets requiring tree partitioning under memory constraints.

ber of leaf trajectories and total tokens constant across settings. This setup allows us to systematically study how the overlap degree influences computation reuse and throughput.

As shown in Figure 8, Tree Training consistently reduces total training time across all conditions, with the improvement magnitude increasing monotonically with the Potential Overlap Ratio (POR). In the ideal setting where full trees fit in memory, Tree Training achieves up to $8.7\times$ speedup by eliminating redundant computation through shared prefix reuse.

4.6 Memory footprint

In our implementation, even when large trees must be split (achieved via Redundancy-Free Tree Partitioning), our method eliminates all redundant computation and our performance closely tracks the theoretical speedup upper bound of $\frac{1}{(1-\text{POR})}$, demonstrating that the gateway mechanism incurs negligible runtime overhead. The additional tensors

required by Tree Training total only **1.2 MB** on Qwen3-32B (Yang et al., 2025a), compared to the 64000 MB minimum activation memory, confirming negligible memory overhead.

4.7 The gain of training on all tokens of trajectory trees

Training on all tokens of trajectory trees, compared to training on only a single trajectory (in our baseline experiment, we select the longest trajectory as common practice), leads to a clear performance gain, especially for thinking models.

On Terminal Bench 2.0 (Merrill et al., 2026), we use a cold-started Qwen3-32B (Yang et al., 2025a) as the base checkpoint for RL training. The model trained on the full tree (with think-mode turned on) achieves a score (avg@4) of 28.8, whereas the baseline score is 20.9.

5 CONCLUSION

We propose Tree Training, a performance optimization framework during the training phase. It aims to minimize redundant computation and boost efficiency while strictly maintaining the gradient values of the parameters undergoing optimization. In contrast to existing prefix reuse schemes, our Tree Training is not only suitable when the prefix is just a prompt that does not contribute to gradients, but also when the prefix is involved in gradient updates.

REFERENCES

- claude-code. URL <https://claude.com/product/claude-code>.
- Abdelaziz, I., Basu, K., Agarwal, M., Kumaravel, S., Stallone, M., Panda, R., Rizk, Y., Bhargav, G. P. S., Crouse, M., Gunasekara, C., Ikbali, S., Joshi, S., Karanam, H., Kumar, V., Munawar, A., Neelam, S., Raghu, D., Sharma, U., Soria, A. M., Sreedhar, D., Venkateswaran, P., Unuvar, M., Cox, D. D., Roukos, S., Lastras, L. A., and Kapanipathi, P. Granite-function calling model: Introducing function calling abilities via multi-task learning of granular tasks. In Dernoncourt, F., Preotiu-Pietro, D., and Shimorina, A. (eds.), *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pp. 1131–1139, Miami, Florida, US, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-industry.85. URL <https://aclanthology.org/2024.emnlp-industry.85/>.
- Cheng, Y., Du, K., Yao, J., and Jiang, J. Do large language models need a content delivery network? *arXiv preprint arXiv:2409.13761*, 2024.
- Cheng, Y., Liu, Y., Yao, J., An, Y., Chen, X., Feng, S., Huang, Y., Shen, S., Du, K., and Jiang, J. Lmcache: An efficient kv cache layer for enterprise-scale llm inference. *arXiv preprint arXiv:2510.09665*, 2025.
- Dao, T. and Gu, A. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality, 2024. URL <https://arxiv.org/abs/2405.21060>.
- DeepSeek-AI, Liu, A., Feng, B., Wang, B., Wang, B., Liu, B., Zhao, C., Dengr, C., Ruan, C., Dai, D., Guo, D., Yang, D., Chen, D., Ji, D., Li, E., Lin, F., Luo, F., Hao, G., Chen, G., Li, G., Zhang, H., Xu, H., Yang, H., Zhang, H., Ding, H., Xin, H., Gao, H., Li, H., Qu, H., Cai, J. L., Liang, J., Guo, J., Ni, J., Li, J., Chen, J., Yuan, J., Qiu, J., Song, J., Dong, K., Gao, K., Guan, K., Wang, L., Zhang, L., Xu, L., Xia, L., Zhao, L., Zhang, L., Li, M., Wang, M., Zhang, M., Zhang, M., Tang, M., Li, M., Tian, N., Huang, P., Wang, P., Zhang, P., Zhu, Q., Chen, Q., Du, Q., Chen, R. J., Jin, R. L., Ge, R., Pan, R., Xu, R., Chen, R., Li, S. S., Lu, S., Zhou, S., Chen, S., Wu, S., Ye, S., Ma, S., Wang, S., Zhou, S., Yu, S., Zhou, S., Zheng, S., Wang, T., Pei, T., Yuan, T., Sun, T., Xiao, W. L., Zeng, W., An, W., Liu, W., Liang, W., Gao, W., Zhang, W., Li, X. Q., Jin, X., Wang, X., Bi, X., Liu, X., Wang, X., Shen, X., Chen, X., Chen, X., Nie, X., Sun, X., Wang, X., Liu, X., Xie, X., Yu, X., Song, X., Zhou, X., Yang, X., Lu, X., Su, X., Wu, Y., Li, Y. K., Wei, Y. X., Zhu, Y. X., Xu, Y., Huang, Y., Li, Y., Zhao, Y., Sun, Y., Li, Y., Wang, Y., Zheng, Y., Zhang, Y., Xiong, Y., Zhao, Y., He, Y., Tang, Y., Piao, Y., Dong, Y., Tan, Y., Liu, Y., Wang, Y., Guo, Y., Zhu, Y., Wang, Y., Zou, Y., Zha, Y., Ma, Y., Yan, Y., You, Y., Liu, Y., Ren, Z. Z., Ren, Z., Sha, Z., Fu, Z., Huang, Z., Zhang, Z., Xie, Z., Hao, Z., Shao, Z., Wen, Z., Xu, Z., Zhang, Z., Li, Z., Wang, Z., Gu, Z., Li, Z., and Xie, Z. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model, 2024. URL <https://arxiv.org/abs/2405.04434>.
- Goru, R., Mehta, S., and Jain, P. One-pass to reason: Token duplication and block-sparse mask for efficient fine-tuning on multi-turn reasoning, 2025. URL <https://arxiv.org/abs/2504.18246>.
- Gu, A. and Dao, T. Mamba: Linear-time sequence modeling with selective state spaces, 2024. URL <https://arxiv.org/abs/2312.00752>.
- Gupta, N., Wang, P., Kannan, R., and Prasanna, V. K. A persistent-state dataflow accelerator for memory-bound linear attention decode on FPGA, 2026. URL <https://arxiv.org/abs/2603.05931>.
- Hou, Z., Hu, Z., Li, Y., Lu, R., Tang, J., and Dong, Y. Treerl: Llm reinforcement learning with on-policy tree search, 2025. URL <https://arxiv.org/abs/2506.11902>.
- Ji, Y., Ma, Z., Wang, Y., Chen, G., Chu, X., and Wu, L. Tree search for llm agent reinforcement learning, 2025. URL <https://arxiv.org/abs/2509.21240>.
- Kim, S., Moon, S., Tabrizi, R., Lee, N., Mahoney, M. W., Keutzer, K., and Gholami, A. An llm compiler for parallel function calling, 2024. URL <https://arxiv.org/abs/2312.04511>.
- Krell, M. M., Kosec, M., Perez, S. P., and Fitzgibbon, A. Efficient sequence packing without cross-contamination: Accelerating large language models without impacting performance. *arXiv preprint arXiv:2107.02027*, 2021.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention, 2023. URL <https://arxiv.org/abs/2309.06180>.
- Li, Y., Gu, Q., Wen, Z., Li, Z., Xing, T., Guo, S., Zheng, T., Zhou, X., Qu, X., Zhou, W., Zhang, Z., Shen, W., Liu, Q., Lin, C., Yang, J., Zhang, G., and Huang, W. Treepo: Bridging the gap of policy optimization and efficacy and inference efficiency with heuristic tree-based modeling, 2025. URL <https://arxiv.org/abs/2508.17445>.

- Liu, Y., Li, H., Cheng, Y., Ray, S., Huang, Y., Zhang, Q., Du, K., Yao, J., Lu, S., Ananthanarayanan, G., et al. Cachegen: Kv cache compression and streaming for fast large language model serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*, pp. 38–56, 2024.
- Liu, Z., Yue, T., Tang, Y., Guo, L., Cai, J., Liu, Q., Chen, X., and Liu, J. Prefix grouper: Efficient grpo training through shared-prefix forward, 2025. URL <https://arxiv.org/abs/2506.05433>.
- Luo, X., Zhang, Y., He, Z., Wang, Z., Zhao, S., Li, D., Qiu, L. K., and Yang, Y. Agent lightning: Train any ai agents with reinforcement learning, 2025. URL <https://arxiv.org/abs/2508.03680>.
- Merrill, M. A., Shaw, A. G., Carlini, N., Li, B., Raj, H., Bercovich, I., Shi, L., Shin, J. Y., Walshe, T., Buchanan, E. K., Shen, J., Ye, G., Lin, H., Poulos, J., Wang, M., Nezhurina, M., Jitsev, J., Lu, D., Mastromichalakis, O. M., Xu, Z., Chen, Z., Liu, Y., Zhang, R., Chen, L. L., Kashyap, A., Uslu, J.-L., Li, J., Wu, J., Yan, M., Bian, S., Sharma, V., Sun, K., Dillmann, S., Anand, A., Lanpouthakoun, A., Koopah, B., Hu, C., Guha, E., Dreiman, G. H. S., Zhu, J., Krauth, K., Zhong, L., Muennighoff, N., Amanfu, R., Tan, S., Pimpalgaonkar, S., Aggarwal, T., Lin, X., Lan, X., Zhao, X., Liang, Y., Wang, Y., Wang, Z., Zhou, C., Heineman, D., Liu, H., Trivedi, H., Yang, J., Lin, J., Shetty, M., Yang, M., Omi, N., Raoof, N., Li, S., Zhuo, T. Y., Lin, W., Dai, Y., Wang, Y., Chai, W., Zhou, S., Wahdany, D., She, Z., Hu, J., Dong, Z., Zhu, Y., Cui, S., Saiyed, A., Kolbeinsson, A., Hu, J., Rytting, C. M., Marten, R., Wang, Y., Dimakis, A., Konwinski, A., and Schmidt, L. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces, 2026. URL <https://arxiv.org/abs/2601.11868>.
- Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., and Gonzalez, J. E. Memgpt: Towards llms as operating systems, 2024. URL <https://arxiv.org/abs/2310.08560>.
- Perron, L. and Furnon, V. Or-tools. URL <https://developers.google.com/optimization/>.
- Pope, R., Douglas, S., Chowdhery, A., Devlin, J., Bradbury, J., Levskaya, A., Heek, J., Xiao, K., Agrawal, S., and Dean, J. Efficiently scaling transformer inference, 2022. URL <https://arxiv.org/abs/2211.05102>.
- Qin, R., Li, Z., He, W., Zhang, M., Wu, Y., Zheng, W., and Xu, X. Mooncake: A kvcache-centric disaggregated architecture for llm serving, 2025. URL <https://arxiv.org/abs/2407.00079>.
- Shah, J., Bikshandi, G., Zhang, Y., Thakkar, V., Ramani, P., and Dao, T. Flashattention-3: Fast and accurate attention with asynchrony and low-precision, 2024. URL <https://arxiv.org/abs/2407.08608>.
- Shoeybi, M., Patwary, M., Puri, R., LeGresley, P., Casper, J., and Catanzaro, B. Megatron-lm: Training multi-billion parameter language models using model parallelism, 2020. URL <https://arxiv.org/abs/1909.08053>.
- Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B., and Liu, Y. Roformer: Enhanced transformer with rotary position embedding, 2023. URL <https://arxiv.org/abs/2104.09864>.
- Team, K., Bai, T., Bai, Y., Bao, Y., Cai, S. H., Cao, Y., Charles, Y., Che, H. S., Chen, C., Chen, G., Chen, H., Chen, J., Chen, J., Chen, J., Chen, J., Chen, K., Chen, L., Chen, R., Chen, X., Chen, Y., Chen, Y., Chen, Y., Chen, Y., Chen, Y., Chen, Y., Chen, Y., Chen, Y., Chen, Y., Chen, Y., Chen, Z., Chen, Z., Cheng, D., Chu, M., Cui, J., Deng, J., Diao, M., Ding, H., Dong, M., Dong, M., Dong, Y., Dong, Y., Du, A., Du, C., Du, D., Du, L., Du, Y., Fan, Y., Fang, S., Feng, Q., Feng, Y., Fu, G., Fu, K., Gao, H., Gao, T., Ge, Y., Geng, S., Gong, C., Gong, X., Gongque, Z., Gu, Q., Gu, X., Gu, Y., Guan, L., Guo, Y., Hao, X., He, W., He, W., He, Y., Hong, C., Hu, H., Hu, J., Hu, Y., Hu, Z., Huang, K., Huang, R., Huang, W., Huang, Z., Jiang, T., Jiang, Z., Jin, X., Jing, Y., Lai, G., Li, A., Li, C., Li, C., Li, F., Li, G., Li, G., Li, H., Li, H., Li, J., Li, J., Li, J., Li, L., Li, M., Li, W., Li, W., Li, X., Li, X., Li, Y., Li, Y., Li, Y., Li, Y., Li, Z., Li, Z., Liao, W., Lin, J., Lin, X., Lin, Z., Lin, Z., Liu, C., Liu, C., Liu, H., Liu, L., Liu, S., Liu, S., Liu, S., Liu, T., Liu, T., Liu, W., Liu, X., Liu, Y., Liu, Y., Liu, Y., Liu, Y., Liu, Y., Liu, Y., Liu, Z., Liu, Z., Lu, E., Lu, H., Lu, Z., Luo, J., Luo, T., Luo, Y., Ma, L., Ma, Y., Mao, S., Mei, Y., Men, X., Meng, F., Meng, Z., Miao, Y., Ni, M., Ouyang, K., Pan, S., Pang, B., Qian, Y., Qin, R., Qin, Z., Qiu, J., Qu, B., Shang, Z., Shao, Y., Shen, T., Shen, Z., Shi, J., Shi, L., Shi, S., Song, F., Song, P., Song, T., Song, X., Su, H., Su, J., Su, Z., Sui, L., Sun, J., Sun, J., Sun, T., Sung, F., Tai, Y., Tang, C., Tang, H., Tang, X., Tang, Z., Tao, J., Teng, S., Tian, C., Tian, P., Wang, A., Wang, B., Wang, C., Wang, C., Wang, C., Wang, D., Wang, D., Wang, D., Wang, F., Wang, H., Wang, H., Wang, H., Wang, H., Wang, H., Wang, J., Wang, J., Wang, J., Wang, K., Wang, L., Wang, Q., Wang, S., Wang, S., Wang, S., Wang, W., Wang, X., Wang, X., Wang, Y., Wang, Y., Wang, Y., Wang, Y., Wang, Y., Wang, Y., Wang, Z., Wang, Z., Wang, Z., Wang, Z., Wang, Z., Wang, Z., Wei, C., Wei, M., Wen, C., Wen, Z., Wu, C., Wu, H., Wu, J., Wu, R., Wu, W., Wu, Y., Wu, Y., Wu, Y., Wu, Z., Xiao, C., Xie, J., Xie, X., Xie, Y., Xin, Y., Xing, B., Xu, B., Xu, J., Xu, J., Xu, J., Xu, L. H., Xu, L., Xu, S., Xu, W., Xu, X., Xu, X., Xu, Y., Xu, Y., Xu, Y., Xu, Z., Xu, Z., Yan, J., Yan, Y., Yang, G., Yang, H., Yang, J.,

- Yang, K., Yang, N., Yang, R., Yang, X., Yang, X., Yang, Y., Yang, Y., Yang, Z., Yang, Z., Yang, Z., Yao, H., Ye, D., Ye, W., Ye, Z., Yin, B., Yu, C., Yu, L., Yu, T., Yu, T., Yuan, E., Yuan, M., Yuan, X., Yue, Y., Zeng, W., Zha, D., Zhan, H., Zhang, D., Zhang, H., Zhang, J., Zhang, P., Zhang, Q., Zhang, R., Zhang, X., Zhang, Y., Zhang, Y., Zhang, Y., Zhang, Y., Zhang, Y., Zhang, Y., Zhang, Y., Zhang, Y., Zhang, Y., Zhang, Y., Zhang, Y., Zhao, C., Zhao, F., Zhao, J., Zhao, S., Zhao, X., Zhao, Y., Zhao, Z., Zheng, H., Zheng, R., Zheng, S., Zheng, T., Zhong, J., Zhong, L., Zhong, W., Zhou, M., Zhou, R., Zhou, X., Zhou, Z., Zhu, J., Zhu, L., Zhu, X., Zhu, Y., Zhu, Z., Zhuang, J., Zhuang, W., Zou, Y., and Zu, X. Kimi k2.5: Visual agentic intelligence, 2026. URL <https://arxiv.org/abs/2602.02276>.
- Team, Q. Qwen3.5: Accelerating productivity with native multimodal agents, February 2026. URL <https://qwen.ai/blog?id=qwen3.5>.
- Wang, F. and Hegde, S. Accelerating direct preference optimization with prefix sharing, 2024. URL <https://arxiv.org/abs/2410.20305>.
- Wang, G., Zeng, J., Xiao, X., Wu, S., Yang, J., Zheng, L., Chen, Z., Bian, J., Yu, D., and Wang, H. Flashmask: Efficient and rich mask extension of flashattention, 2025. URL <https://arxiv.org/abs/2410.01359>.
- Xu, W., Mei, K., Gao, H., Tan, J., Liang, Z., and Zhang, Y. A-mem: Agentic memory for llm agents. *arXiv preprint arXiv:2502.12110*, 2025.
- Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., Zheng, C., Liu, D., Zhou, F., Huang, F., Hu, F., Ge, H., Wei, H., Lin, H., Tang, J., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Zhou, J., Lin, J., Dang, K., Bao, K., Yang, K., Yu, L., Deng, L., Li, M., Xue, M., Li, M., Zhang, P., Wang, P., Zhu, Q., Men, R., Gao, R., Liu, S., Luo, S., Li, T., Tang, T., Yin, W., Ren, X., Wang, X., Zhang, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Zhang, Y., Wan, Y., Liu, Y., Wang, Z., Cui, Z., Zhang, Z., Zhou, Z., and Qiu, Z. Qwen3 technical report, 2025a. URL <https://arxiv.org/abs/2505.09388>.
- Yang, J., Lieret, K., Jimenez, C. E., Wettig, A., Khandpur, K., Zhang, Y., Hui, B., Press, O., Schmidt, L., and Yang, D. Swe-smith: Scaling data for software engineering agents, 2025b. URL <https://arxiv.org/abs/2504.21798>.
- Yang, S., Kautz, J., and Hatamizadeh, A. Gated delta networks: Improving mamba2 with delta rule, 2025c. URL <https://arxiv.org/abs/2412.06464>.
- Yao, J., Li, H., Liu, Y., Ray, S., Cheng, Y., Zhang, Q., Du, K., Lu, S., and Jiang, J. Cacheblend: Fast large language model serving for rag with cached knowledge fusion. In *Proceedings of the Twentieth European Conference on Computer Systems*, pp. 94–109, 2025. doi: 10.1145/3689031.3696098.
- Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T., Cao, Y., and Narasimhan, K. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems*, 36:11809–11822, 2023.
- Zhang, D., Zhoubian, S., Hu, Z., Yue, Y., Dong, Y., and Tang, J. Rest-mcts*: Llm self-training via process reward guided tree search. *Advances in Neural Information Processing Systems*, 37:64735–64772, 2024.
- Zhang, H., Liu, X., Lv, B., Sun, X., Jing, B., Iong, I. L., Hou, Z., Qi, Z., Lai, H., Xu, Y., Lu, R., Wang, H., Tang, J., and Dong, Y. Agentrl: Scaling agentic reinforcement learning with a multi-turn, multi-task framework, 2025. URL <https://arxiv.org/abs/2510.04206>.
- Zhong, W., Guo, L., Gao, Q., Ye, H., and Wang, Y. Memorybank: Enhancing large language models with long-term memory, 2023. URL <https://arxiv.org/abs/2305.10250>.
- Zhou, Y., Jiang, S., Tian, Y., Weston, J., Levine, S., Sukhbaatar, S., and Li, X. Sweet-rl: Training multi-turn llm agents on collaborative reasoning tasks, 2025. URL <https://arxiv.org/abs/2503.15478>.

A DFS SERIALIZATION: IMPLEMENTATION DETAILS

A.1 Tree Attention Mask via FlashMask

The tree attention mask allows token i to attend to token j iff j is in an ancestor chunk of i (full access to the entire ancestor chunk) or in the same chunk with $j \leq i$ (causal within chunk). We implement this as a block-sparse mask using FlashMask (Wang et al., 2025), which extends FlashAttention to support rich per-query start/end column bounds and skips masked blocks entirely:

```
attn_mask_bool = torch.zeros(N, N, dtype=torch.bool)
for c in range(n_chunks):
    c_start, c_end = chunk_boundaries[c]
    for anc in ancestors[c]:          # ancestors[c] includes c itself
        a_start, a_end = chunk_boundaries[anc]
        for i in range(c_start, c_end):
            if anc == c:              # causal within own chunk
                attn_mask_bool[i, c_start : i + 1] = True
            else:                     # full access to ancestor chunk
                attn_mask_bool[i, a_start : a_end] = True
```

A.2 Tree-Aware SSM State Routing

Standard chunk GatedDeltaNet passes the recurrent state sequentially: chunk i reads the output state of chunk $i-1$. Under DFS serialization, after a leaf chunk the sequence backtracks to a sibling branch, so the sequential predecessor is a sibling, not an ancestor. We replace sequential routing with tree routing: each chunk reads the output state of its *parent* chunk. `all_states[0]` holds the zero initial state; `all_states[i+1]` holds the state produced after processing chunk i . The index arithmetic `parent_list_idx = chunk_parent_map[i] + 1` maps root chunks (`parent_map=-1`) to index 0 and non-root chunks to their parent’s output state. DFS pre-order guarantees the parent state is always computed before any child requests it. Sibling chunks reference the same parent state tensor; PyTorch autograd accumulates their gradient contributions there automatically.

```
def torch_chunk_gated_delta_rule_tree_varlen(
    query, key, value, g, beta,
    chunk_boundaries,  # list[(start, end)] token index range per tree node
    chunk_parent_map,  # list[int] parent chunk idx; -1 = use initial_state
    initial_state=None,
):
    init = (initial_state.float() if initial_state is not None
            else torch.zeros(B, H, Dk, Dv, dtype=torch.float32, device=dev))

    all_states = [init]  # all_states[0]=init, all_states[i+1]=state after chunk i
    all_outputs = []

    for i, (c_start, c_end) in enumerate(chunk_boundaries):
        L = c_end - c_start
        q_i, k_i, v_i = q[:, :, c_start:c_end], k[:, :, c_start:c_end], v[:, :, c_start:c_end]
        bt_i, g_i = bt[:, :, c_start:c_end], g[:, :, c_start:c_end]

        v_beta_i = v_i * bt_i.unsqueeze(-1)
        k_beta_i = k_i * bt_i.unsqueeze(-1)
        g_cum = g_i.cumsum(dim=-1)  # [B, H, L]
        decay_mask = (g_cum.unsqueeze(-1) - g_cum.unsqueeze(-2)).tril().exp().tril()

        # ---- Within-chunk SSM correction ----
        attn_raw = -(k_beta_i @ k_i.transpose(-1, -2) * decay_mask).masked_fill(
            triu_mask, 0)
        attn_rows = [attn_raw[..., 0:1, :]]
        for j in range(1, L):
            attn_rows.append(attn_raw[..., j:j+1, :] + attn_raw[..., j:j+1, :j] @
                             attn_raw[..., :j, :])
        attn = torch.cat(attn_rows, dim=-2) + torch.eye(L, device=dev)
```

```

value_corr = attn @ v_beta_i # [B, H, L, Dv]
k_cumdecay = attn @ (k_beta_i * g_cum.exp()).unsqueeze(-1)) # [B, H, L, Dk]

# ---- Cross-chunk: read from PARENT state (tree routing) ----
parent_list_idx = int(chunk_parent_map[i]) + 1 # -1->0, j->j+1
current_state = all_states[parent_list_idx] # [B, H, Dk, Dv]

v_prime = k_cumdecay @ current_state # [B, H, L, Dv]
v_new = value_corr - v_prime
attn_within = (q_i @ k_i.transpose(-1,-2) * decay_mask).masked_fill(triu_excl,
0)
attn_inter = (q_i * g_cum.exp()).unsqueeze(-1)) @ current_state
out_i = attn_inter + attn_within @ v_new # [B, H, L, Dv]
all_outputs.append(out_i)

# ---- Store new state for children ----
new_state = (
    current_state * g_cum[:, :, -1, None, None].exp()
    + (k_i * (g_cum[:, :, -1:] - g_cum).exp()).unsqueeze(-1)).transpose(-1,-2) @
    v_new
)
all_states.append(new_state) # sibling chunks will both read current_state
above

core_out = torch.cat(all_outputs, dim=2).transpose(1, 2) # [B, S, H, Dv]
return core_out

```

A.3 Tree-Correct Causal Convolution

GDN layers include a causal conv1d with kernel size K_{conv} . At a DFS backtrack position, the $K_{\text{conv}}-1$ tokens immediately before a child chunk belong to a sibling branch, not the ancestor path, so using them as context is incorrect. We fix this by having each chunk prepend its *parent*'s saved last $K_{\text{conv}}-1$ effective tokens as left context, then discard them from the output. The saved context is taken from `ext[:, :, -(K-1):]`, i.e. the tail of the concatenated `[parent_ctx, chunk]` tensor—which includes ancestor history—so grandchildren see the full $K_{\text{conv}}-1$ ancestor token history, matching an independent per-path forward.

```

def _tree_correct_conv_varlen(ssm, mixed_qkv_raw, chunk_boundaries, chunk_parent_map):
    K = ssm.conv_kernel_size
    chunk_outputs = []
    conv_states = {} # chunk_idx -> [B, conv_dim, K-1] saved context for children

    for c, (c_start, c_end) in enumerate(chunk_boundaries):
        L = c_end - c_start
        chunk = mixed_qkv_raw[:, :, c_start:c_end] # [B, conv_dim, L]

        parent_state = conv_states.get(chunk_parent_map[c], None)

        if parent_state is not None:
            ext = torch.cat([parent_state, chunk], dim=-1) # [B, conv_dim, K-1+L]
            out = ssm.causal_conv1d_fn(
                x=ext, weight=ssm.conv1d.weight.squeeze(1),
                bias=ssm.conv1d.bias, activation=ssm.activation)
            chunk_out = out[:, :, K-1:] # drop prepended context -> [B, conv_dim, L]
        else:
            chunk_out = ssm.causal_conv1d_fn(
                x=chunk, weight=ssm.conv1d.weight.squeeze(1),
                bias=ssm.conv1d.bias, activation=ssm.activation)

        chunk_outputs.append(chunk_out)

    # Save last K-1 EFFECTIVE tokens (ancestor ctx + chunk tail) for children.

```



```

    # Using ext tail (not just chunk) propagates ancestor history to grandchildren.
    if parent_state is not None:
        conv_states[c] = ext[:, :, -(K - 1):].clone()
    else:
        conv_states[c] = F.pad(chunk, (max(0, K-1-L), 0))[:, :, -(K-1):].clone()

    return torch.cat(chunk_outputs, dim=2) # [B, conv_dim, N]

```

B ZERO-OVERHEAD PARTITION BOUNDARIES: IMPLEMENTATION DETAILS

This appendix provides the complete implementation details for the KV-cache gradient checkpointing at partition boundaries described in Section 3.3.

B.1 Data Structures

SubtreeGateway. Each cut node n_c in partition P produces one gateway for the child partition:

- $\{(\tilde{\mathbf{G}}_{n_c}^{(l)}, \tilde{\mathbf{V}}_{n_c}^{(l)})\}_l$: detached leaf KV tensors with `requires_grad=True`, shape $[1, H, e_{n_c}, d]$ per layer.
- $\{(\mathbf{G}_{n_c}^{(l)}, \mathbf{V}_{n_c}^{(l)})\}_l$: original KV slices retaining `grad_fn`.
- Float32 accumulator hooks: registered on each leaf tensor to capture gradient contributions in float32 before bfloat16 rounding; see below.
- `attn_bias` $\in \mathbb{R}^{e_{n_c}}$: additive attention bias with 0 at ancestor positions and $-\infty$ at non-ancestor positions (Section B.3).
- `pos_offset`: depth-based position of the child’s first token (Section B.4).
- For SSM hybrid: $\{(\tilde{\mathbf{S}}_{n_c}^{(l)}, \tilde{\mathbf{C}}_{n_c}^{(l)})\}_l$ and their raw counterparts (Section B.7).

B.2 Forward Pass

Parent partition forward. Run DFS forward on P ’s token sequence with the standard tree attention mask and SSM patch (Section 3.2). The model is called with `past_key_values = GatewayCache( $\tilde{\mathbf{G}}_{\text{parent}}$ )` if P itself is a child partition, otherwise an empty cache.

After the forward, for each cut node $n_c \in P$:

1. Slice the accumulated KV: $\mathbf{G}_{n_c}^{(l)} \leftarrow \mathbf{K}_{[:, \text{past_len} + e_{n_c}]}$, which retains `grad_fn` via the concatenation inside GatewayCache.
2. Detach to create gateway leaf tensors and register float32 accumulator hooks:

$$\tilde{\mathbf{G}}_{n_c}^{(l)} = \text{detach}(\mathbf{G}_{n_c}^{(l)}) . \text{requires_grad}(\text{True}). \quad (13)$$

Child partition forward. Run DFS forward on C ’s own token sequence with the *combined* attention mask:

$$M_{ij} = \begin{cases} \text{attn_bias}_j & j < \text{past_len} \\ m_{i, j - \text{past_len}}^{\text{tree}} & j \geq \text{past_len} \end{cases} \quad (14)$$

where m^{tree} is the standard within-subtree tree attention mask. The model receives `past_key_values = GatewayCache( $\tilde{\mathbf{G}}_{n_c}$ )`.

B.3 Ancestor-Aware Attention Bias

In a DFS-ordered partition, the gateway slice $[: e_{n_c}]$ may include tokens from sibling subtrees that are *not* ancestors of n_c . For example, if P 's DFS sequence is $[a, b, c_1, c_2]$ where c_1, c_2 are siblings and $n_c = c_2$, the gateway includes c_1 's tokens, which n_c 's children must not attend to. We compute ancestor positions by traversing n_c 's parent chain within P :

$$\text{anc}(n_c, P) = \bigcup_{n' \in \text{path}(P.\text{root}, n_c)} [s_{n'}, e_{n'}] \quad (15)$$

where $[s_{n'}, e_{n'}]$ is the token range of n' in P 's DFS sequence. The attention bias is:

$$\text{attn_bias}_j = \begin{cases} 0 & j \in \text{anc}(n_c, P) \cup [0, \text{past_len}_P) \\ -\infty & \text{otherwise} \end{cases} \quad (16)$$

(The term $[0, \text{past_len}_P)$ covers P 's own gateway past, which is always fully visible since it was already ancestor-filtered when P 's gateway was created.)

B.4 Depth-Based Position Offset

The child partition's position IDs must match those that n_c 's children would receive in a standalone per-path forward (to ensure RoPE produces identical results). Let $\text{pos}_{n_c}^{(P)}$ be the depth-based position of n_c 's *first* token within P (obtained from P 's `dfs_pos_ids` array). Then:

$$\text{pos_offset}(C) = \text{pos_offset}(P) + \text{pos}_{n_c}^{(P)} + |T_{n_c}| \quad (17)$$

Critically, $\text{pos_offset}(C) \neq \text{past_len}$ when P 's DFS contains sibling tokens between P 's first token and n_c : `past_len` counts all gateway tokens (including non-ancestor siblings), whereas (17) counts only ancestor tokens up to and including n_c .

B.5 Float32 Gradient Accumulation

When multiple child partitions share the same cut node (i.e., n_c has $k > 1$ children going to distinct child partitions), all children's backward passes accumulate gradients into the same leaf gateway tensors. Direct accumulation into the `bfloat16.grad` attribute introduces rounding errors from repeated `bfloat16` additions. We register a `float32` accumulator hook on each leaf tensor:

$$\tilde{\mathbf{G}}^{(l)}.register_hook(\lambda g : \text{acc}^{(l)} += g.\text{float}(), \text{return } \mathbf{0}) \quad (18)$$

Returning `0` suppresses writes to the `bfloat16.grad`, so the `float32` accumulator captures the exact sum. The chain backward (19) then uses the `float32` accumulated values. Note: even though we pass `float32` gradients to `autograd.backward`, PyTorch's type-promotion rules cast them back to `bfloat16` when entering a `bfloat16` `matmul`; the hook therefore primarily eliminates *inter-child accumulation* rounding, not the in-kernel computation.

B.6 Backward Pass and retain_graph Scheduling

Processing partition P in reverse topological order:

1. For each actual leaf $\ell \in P$: compute $\mathcal{L}_\ell$ and call $\mathcal{L}_\ell.\text{backward}(\text{retain_graph} = \text{True})$.
2. For each unique cut node n_c of P (deduplicated to avoid double-counting when n_c has multiple children): chain gradients back via:

$$\text{autograd.backward}\left(\left\{\mathbf{G}_{n_c}^{(l)}\right\}_l, \left\{\nabla_{\tilde{\mathbf{G}}_{n_c}^{(l)}}\right\}_l\right) \quad (19)$$

with `retain_graph = True` for all but the last operation, `False` for the last.

Total backward traversals of P 's computation graph = $|\{\ell \in P : \ell \text{ is a leaf}\}| + |\text{unique cut nodes of } P|$. The graph is freed after the last traversal.

B.7 SSM State Gateway

State extraction. The existing DFS SSM forward (`torch_chunk_gated_delta_rule_tree_varlen`) maintains an `all_states` list where `all_states[i+1]` is the recurrent state after chunk i . We extend it with a `capture_states_at` argument: after computing `new_state` for chunk c , if $c \in \text{capture_states_at}$, store `all_states[c+1]` (which has `grad_fn`). Similarly, `_tree_correct_conv_varlen` is extended to return `conv_states[c]` (the $K-1$ conv context after chunk c) at requested positions.

State injection. For the child partition’s SSM forward, we inject gateway states via two mechanisms:

- *Recurrent state:* pass $\tilde{\mathbf{S}}_{n_c}^{(l)}$ as `initial_state` to `torch_chunk_gated_delta_rule_tree_varlen`. Root chunks (those with `parent_map = -1`) read from `all_states[0] = initial_state`, matching the gateway.
- *Conv context:* pre-populate `conv_states[-1] =  $\tilde{\mathbf{C}}_{n_c}^{(l)}$` before calling `_tree_correct_conv_varlen`. Root chunks look up `conv_states.get(-1, None)` as their parent context, finding the gateway value.

This design requires no structural changes to the SSM forward; the injection is entirely through the existing lookup mechanism.

Gradient chaining. The raw tensors $\mathbf{S}_{n_c}^{(l)}$ and $\mathbf{C}_{n_c}^{(l)}$ (with `grad_fn`) are included in the `autograd.backward` call alongside the KV tensors. All KV and SSM gradients are chained in a single call per cut node, with a shared `retain_graph` flag.

B.8 Numerical Verification

Pure-attention models (Qwen3-4B).

- *Self-consistency:* two runs with the same inputs give EXACT 0.
- *Backward vs. reference (float32):* max-relative error $< 10^{-4}$ across max-tokens $\in \{1000, 20, 10\}$ (verified against `tree_grad_ckpt_sequential`).

SSM hybrid models (Qwen3.5, 2-layer float32).

- *Self-consistency:* EXACT 0 across all max-tokens settings.
- *Backward vs. reference (float32):* max-relative error $< 2 \times 10^{-5}$, consistent across no-cut (max-tokens = 1000) and aggressively-cut (max-tokens = 3) settings, confirming that the SSM state gateway introduces no additional numerical error beyond standard float32 arithmetic.
- *Forward:* differences of $\sim 5 \times 10^{-6}$ between partition sizes arise from different DFS sequence lengths feeding into float32 operations; two runs with identical partition structure give EXACT 0.